

❌ Switch Statement (Selection Control Structure)

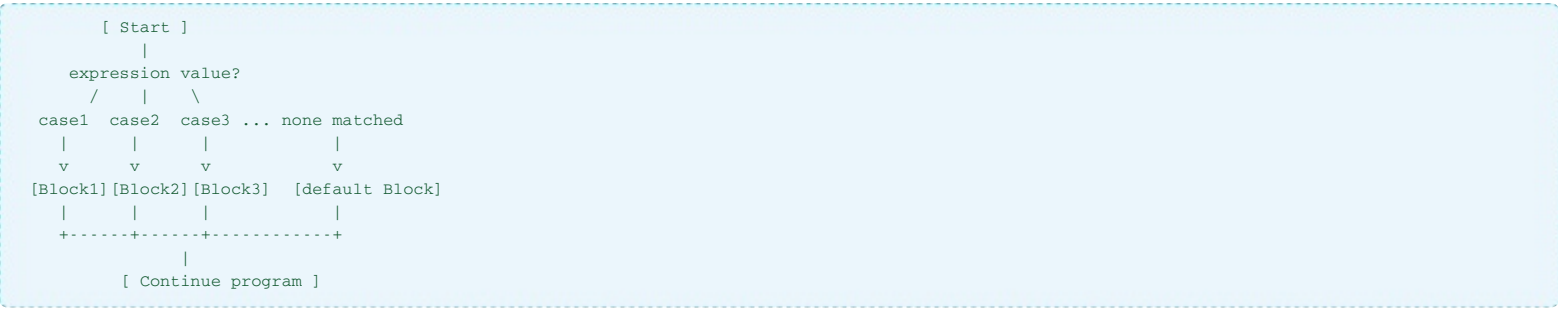
📖 Definition

The **switch** statement is a **selection control structure** used to choose **one block of code** to execute out of many, by matching a variable's value against a list of **constant values (cases)**. It is a cleaner alternative to a long else...if ladder when comparing **one variable** against **many fixed values**.

⚙️ Syntax

```
switch (expression) {
    case value1:
        // statements
        break;
    case value2:
        // statements
        break;
    default:
        // statements if no case matches
}
```

❌ Flow of Control



🔍 How It Works

- The switch expression is evaluated **once**, and its value is compared with each case label, **in order**.
- When a matching case is found, execution starts from **that point** and continues until a break is met.
- The break statement **exits the switch block** — without it, execution "falls through" into the **next case** automatically.
- The default case runs if **none of the cases match** — it is optional and can be placed anywhere, but is usually written last.

📄 Example Program

```
// Program to print the day name using switch
#include <stdio.h>

int main() {
    int day = 3;

    switch (day) {
        case 1: printf("Monday\n");    break;
        case 2: printf("Tuesday\n");   break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n");  break;
        default: printf("Invalid day\n");
    }

    return 0;
}
```

▶️ Output

Wednesday

📄 Example: What Happens Without break (Fall-through)

```
int day = 2;
switch (day) {
    case 1: printf("Monday\n");
    case 2: printf("Tuesday\n");
    case 3: printf("Wednesday\n");
    default: printf("Done\n");
}
```

▶️ Output (no break used!)

Tuesday
Wednesday
Done

Since there is **no break**, execution "falls through" and keeps running **every case below the matched one** too.

📌 Note

- switch works only with **integer or character** values (not float, double, or strings).
- Case labels must be **constant values** — you cannot use a variable or a range like case 1...5.
- Two case labels cannot have the **same value**.
- Always remember to add break, unless fall-through is **intentionally** desired.

🎉 Fun Tip

Forgot **break**? The switch keeps sliding down like a real slide! 🛼

for Loop

Definition

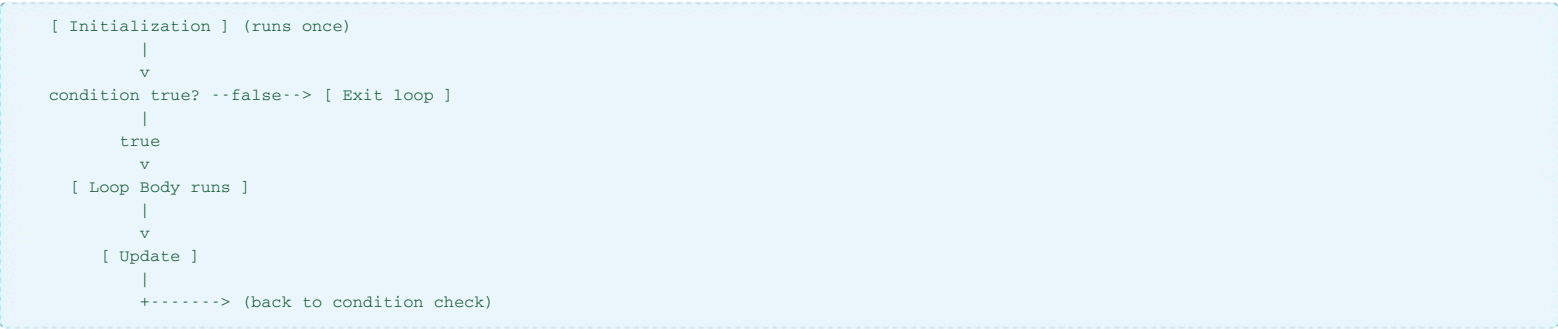
A **loop** is used to **repeat a block of statements** multiple times, instead of writing the same code again and again. The **for loop** is used when you know **in advance how many times** you want to repeat something — it combines **initialization, condition, and update** all in one line.

Syntax

```
for (initialization; condition; update) {  
    // statements to repeat  
}
```

Part	Purpose	Runs
Initialization	Sets the starting value of the loop counter	Only once , at the very start
Condition	Checked before every repetition; loop continues while true	Every time , before each iteration
Update	Changes the counter (increment/decrement)	Every time , after each iteration

Flow of Control



Example Program

```
// Program to print numbers from 1 to 5
#include <stdio.h>

int main() {
    int i;

    for (i = 1; i <= 5; i++) {
        printf("%d ", i);
    }

    return 0;
}
```

Output

1 2 3 4 5

Step-by-Step: How the Loop Runs

- Initialization:** `i = 1` (runs only once).
- Check condition:** `i <= 5` → `1 <= 5` → true → print **1**.
- Update:** `i++` → `i` becomes **2**. Condition checked again → true → print **2**.
- This repeats until `i` becomes **6**, where `6 <= 5` is **false** — loop stops.

Note

- Use a for loop when the **number of repetitions is known** or easily calculated in advance.
- All three parts (initialization, condition, update) are **optional** — `for(;;)` is valid and creates an infinite loop.
- Multiple variables can be initialized/updated using a comma: `for(i=0, j=10; i<j; i++, j--)`

Fun Tip

for = "I know exactly how many times to repeat!" 🧐

🔄 while Loop

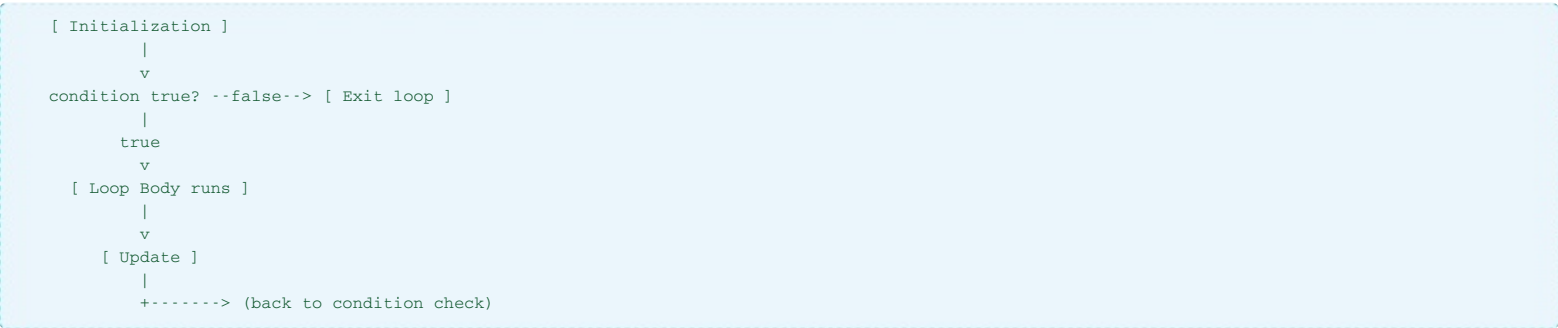
📖 Definition

The **while loop** repeats a block of statements **as long as a given condition remains true**. It is an **entry-controlled loop** — the condition is checked **before** the loop body runs, so the body may execute **zero or more times**.

🔧 Syntax

```
initialization;
while (condition) {
    // statements to repeat
    update;
}
```

⚙️ Flow of Control



🔗 Example Program

```
// Program to print numbers from 1 to 5 using while loop
#include <stdio.h>

int main() {
    int i = 1;

    while (i <= 5) {
        printf("%d ", i);
        i++;
    }

    return 0;
}
```

▶️ Output

1 2 3 4 5

🔑 Key Difference: for vs while

Aspect	for loop	while loop
Best used when	Number of iterations is known	Number of iterations is unknown , depends on a condition
Structure	Init, condition, update all in one line	Init is outside , update is inside the body
Example use	Printing 1 to 100	Reading input until user enters -1

🔗 Example: Loop That Depends on User Input

```
// Keep asking until user enters 0
int num;
printf("Enter a number (0 to stop): ");
scanf("%d", &num);

while (num != 0) {
    printf("You entered: %d\n", num);
    printf("Enter a number (0 to stop): ");
    scanf("%d", &num);
}
```

Here we **can't predict** how many times the loop will run — it depends entirely on when the user enters 0. This is exactly where while shines.

📌 Note

- If the condition is **false the very first time**, the loop body **never executes** — not even once.
- Always make sure something inside the loop **eventually makes the condition false**, or you'll create an **infinite loop**.
- `while(1)` is a common way to write an **intentional** infinite loop (usually combined with a `break` inside).

💡 Fun Tip

`while` = "Keep going while this stays true – checked before every round!"

do...while Loop

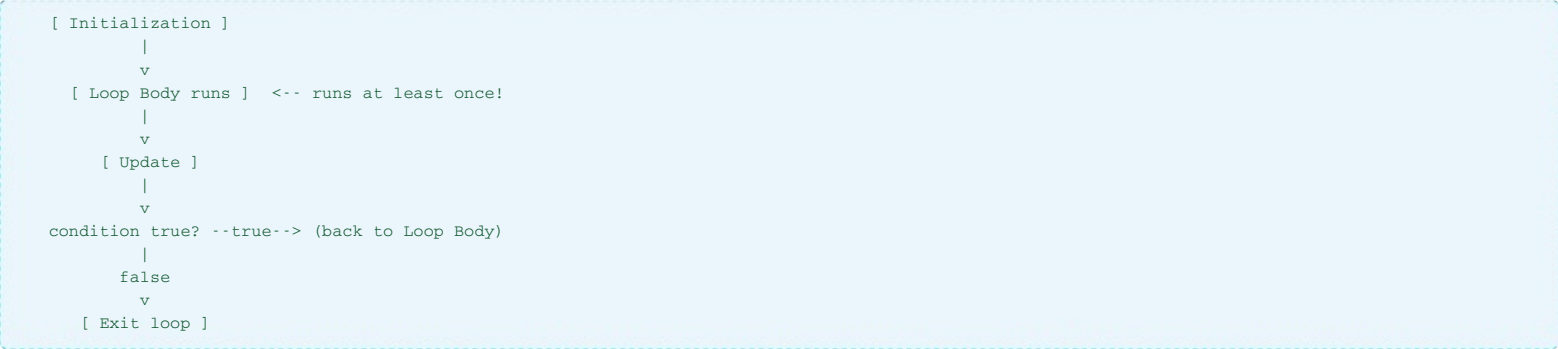
Definition

The **do...while loop** is similar to the while loop, but it is an **exit-controlled loop** — the condition is checked **after** the loop body runs. This guarantees the loop body executes **at least once**, no matter what the condition is.

Syntax

```
initialization;
do {
    // statements to repeat
    update;
} while (condition); // note the semicolon!
```

Flow of Control



Example Program

```
// Program to print numbers from 1 to 5 using do-while loop
#include <stdio.h>

int main() {
    int i = 1;

    do {
        printf("%d ", i);
        i++;
    } while (i <= 5);

    return 0;
}
```

Output

1 2 3 4 5

Example: The "At Least Once" Behaviour

```
int i = 10;

while (i < 5) {
    // condition is FALSE from the start
    printf("while: %d\n", i);
}
// Nothing is printed here — while never runs

do {
    printf("do-while: %d\n", i); // runs once even though condition is false
} while (i < 5);
```

Output

do-while: 10

Even though `i < 5` is **false**, the do-while loop still runs **once** before checking the condition — that's the key difference.

while vs do...while

Aspect	while	do...while
Condition checked	Before the loop body	After the loop body
Minimum executions	0 times	1 time (always)
Ends with	No semicolon after <code>{ }</code>	Semicolon required after <code>while(condition);</code>
Common use	General repetition	Menu-driven programs, input validation

Note

- Don't forget the **semicolon ;** at the end of `while(condition);` in a do-while loop — it's a very common beginner mistake.
- do...while is ideal for situations like **menus**, where you want to show options **at least once** before checking whether to repeat.

Fun Tip

do-while = "Do it first, ask questions later!" 😊

break Statement

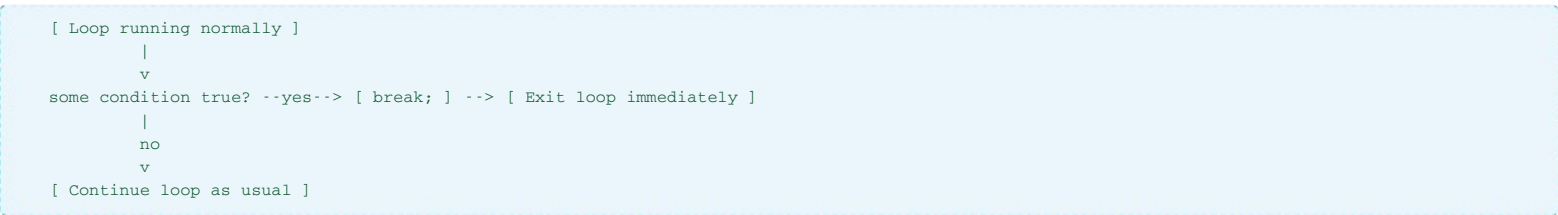
Definition

The **break** statement is used to **immediately exit** a loop (for, while, do-while) or a switch statement, **regardless of the loop's condition**. Control jumps to the **first statement after** the loop or switch block.

Syntax

```
break;
```

Flow of Control (inside a loop)



Example Program

```
// Program to find the first number divisible by 7 between 1 and 50
#include <stdio.h>

int main() {
    int i;

    for (i = 1; i <= 50; i++) {
        if (i % 7 == 0) {
            printf("First number divisible by 7: %d\n", i);
            break;    // stop the loop right away
        }
    }

    return 0;
}
```

Output

```
First number divisible by 7: 7
```

Without break, the loop would keep running all the way to `i = 50`, wasting time checking numbers we no longer care about.

Note

- break exits **only the innermost** loop or switch it is written in — it does **not** affect any outer loops.
- It is commonly used to **stop searching** once the answer is found, saving unnecessary iterations.
- We already saw break used in the switch statement to prevent fall-through — its role there is exactly the same: "stop here, don't continue."

 **Fun Tip**

break = an **emergency exit door** 🚪 out of a loop or switch!

➡➡ continue Statement

📖 Definition

The **continue** statement is used to **skip the rest of the current iteration** of a loop and jump directly to the **next iteration**. Unlike break, it does **not** exit the loop — the loop keeps running.

🔗 Syntax

```
continue;
```

⚡ Flow of Control (inside a loop)

```
[ Loop running normally ]
    |
    v
some condition true? --yes--> [ continue; ] --> (skip to next iteration's condition check)
    |
    no
    v
[ Rest of loop body runs normally ]
```

📄 Example Program

```
// Program to print numbers 1 to 10, skipping even numbers
#include <stdio.h>

int main() {
    int i;

    for (i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue; // skip printing even numbers
        }
        printf("%d ", i);
    }

    return 0;
}
```

▶ Output

```
1 3 5 7 9
```

When i is even, continue skips the printf line and jumps straight to the **update step** (i++), then re-checks the condition.

🔗 break vs continue

Aspect	break	continue
Effect	Exits the loop completely	Skips only the current iteration
Loop continues?	No	Yes , from the next iteration
Used in	Loops and <code>switch</code>	Loops only

📌 Note

- In a for loop, continue still triggers the **update** part (e.g. i++) before checking the condition again.
- In a while or do-while loop, be careful — if the **update statement** is written **after** the continue, it will be skipped too, which can cause an **infinite loop**.
- Use continue when you want to **skip specific cases** but still keep looping through the rest.

💡 Fun Tip

continue = "Skip this round, but keep playing the game!" 🎮

goto Statement

Definition

The **goto** statement transfers control **directly** to a **labeled statement** anywhere else in the same function — jumping forward or backward, skipping the normal top-to-bottom flow entirely.

Syntax

```
goto label;
....
....
label:
    statement;
```

- **label** is simply an **identifier** (a name) followed by a colon :, placed before the statement you want to jump to.
- goto can jump **forward** (skip ahead) or **backward** (repeat earlier code) within the **same function**.

Example Program

```
// Program to print numbers 1 to 5 using goto (loop-like behaviour)
#include <stdio.h>

int main() {
    int i = 1;

    start:           // this is the label
    if (i <= 5) {
        printf("%d ", i);
        i++;
        goto start; // jump back to the label
    }

    return 0;
}
```

Output

```
1 2 3 4 5
```

Step-by-Step: How goto Behaves Here

1. Control reaches the start: label. $i \leq 5 \rightarrow \text{true} \rightarrow$ prints **1**, then i becomes **2**.
2. goto start; jumps **back up** to the label — **not** down to return 0;.
3. This repeats until i becomes **6**, where the if condition becomes false, so execution falls through and the program ends.

Note

- goto is generally **discouraged** in modern programming because it can make code flow **confusing and hard to trace** — this is often called writing "spaghetti code."
- Almost anything done with goto can be done more clearly using if, loops, break, or continue.
- It is mostly seen today only for **breaking out of deeply nested loops** in one step, or in very old legacy code.
- The label used with goto must be inside the **same function** — you cannot jump into another function.

Fun Tip

goto = a "teleporter" \$ inside your code – powerful, but easy to misuse!